Bharatiya Vidya Bhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri (W), Mumbai – 400 058.
Department of Computer Engineering
Lab Assignment

Title: Evaluation of Classical and Post-Quantum Digital Signature Algorithms using Docker-Based Client–Server Communication

1. Aim

To implement the complete digital-signature workflow (key generation, signing, transmission, verification and tamper detection) between two **Docker containers** acting as **sender** and **receiver**, for two classical algorithms (**RSA-PSS** and **ECDSA**) and two NIST-standardized post-quantum algorithms (**ML-DSA** and **SLH-DSA**), and to experimentally compare their key sizes, signature sizes, speed and communication overhead. The experiment also studies **why post-quantum signatures became necessary**.

2. Learning Outcomes

After completing this experiment, students will be able to:

- Draw and explain the digital-signature model with correct naming and notation.
- Build a multi-container environment using Docker images, user-defined networks and containers.
- Generate public/private key pairs for RSA-PSS, ECDSA, ML-DSA and SLH-DSA.
- Sign a document and verify the signature in a separate container after transfer over the network.
- Experimentally demonstrate tampering detection (modified document, modified signature, wrong key).
- Measure key-generation, signing and verification time with a sound benchmarking method.
- Compare key size, signature size and total communication overhead of classical and post-quantum schemes.
- Explain the quantum threat to RSA/ECDSA and the different mathematical bases of ML-DSA and SLH-DSA.
- Justify which algorithm is more suitable for a given application.

3. Algorithms

Students shall implement the signing/verification workflow for:

Category	Algorithm	Standard	Parameter set to use	Security basis	Library
Classical	RSA-PSS	RFC 8017, FIPS 186-5	RSA-3072, e = 65537, SHA-256, MGF1-SHA-256, salt = 32 bytes	Integer factorisation	cryptography
Classical	ECDSA	FIPS 186-5	P-256 (secp256r1), SHA-256	Elliptic-curve discrete log	cryptography
Post-quantum	ML-DSA	FIPS 204 (2024)	ML-DSA-44	Module lattices (Module-LWE / SIS)	liboqs-python (oqs)
Post-quantum	SLH-DSA	FIPS 205 (2024)	SLH-DSA-SHA2-128s (pure)	Hash functions only (hypertree of Merkle/FORS trees)	liboqs-python (oqs)

Note: Students must **use library APIs**; implementing the mathematics of these algorithms from scratch is not expected. ElGamal, Schnorr and DSA are covered in theory only. DSA is retained by FIPS 186-5 only for verifying old signatures, so it is not used in this lab. The chosen parameter sets give roughly comparable (about 128-bit class) security so that the comparison is fair.

4. Background (read before the lab)

A digital signature scheme consists of three algorithms: **KeyGen** → (public key *PK*, private key *SK*); **Sign**(*SK*, *M*) → signature *S*; **Verify**(*PK*, *M*, *S*) → accept / reject. It provides **integrity**, **authentication** of the signer and **non-repudiation**. In practice a long message is first hashed and the digest is signed ("hash-and-sign").

RSA and ECDSA rely on factoring and the elliptic-curve discrete-logarithm problem. **Shor's algorithm**, running on a sufficiently large fault-tolerant quantum computer, would solve both efficiently, so these signatures would become forgeable. Hash functions are only weakened quadratically by Grover's algorithm, and lattice problems have no known efficient quantum attack. In August 2024 NIST finalised **ML-DSA** (lattice-based, derived from CRYSTALS-Dilithium) and **SLH-DSA** (hash-based, derived from SPHINCS+) as post-quantum signature standards. Signatures that must stay trustworthy for decades (firmware, root certificates, legal archives) are the first concern.

5. UID-Based Individualization

To make every student's experiment unique, a number of values are derived from the student's **10-digit UID**. Example: UID = 2024300020. The UID is supplied to the containers as the environment variable `STUDENT_UID`.

Item	Derivation	Used in
ECDSA private key	d = (int(SHA-256(UID || "CNS-DSIG-ECDSA-KEY")) mod (n − 1)) + 1, where n is the order of the P-256 group	keygen.py (ECDSA)
Document content	Line 1: `Transfer Rs.<A> to Account <last 4 digits of UID>` with A = 1000 + (UID mod 9000). Line 2: `Issued by UID: <UID>`	sign.py
Benchmark message	1024 bytes produced by `random.Random(int(UID)).randbytes(1024)`	bench.py
Tampering position	index = UID mod (length of document or signature in bytes)	tamper.py
Result stamp	UID written in the first column/header of every result file	bench.py, report

The order of the P-256 group is `n = FFFFFFFF00000000FFFFFFFFFFFFFFFF BCE6FAADA7179E84F3B9CAC2FC632551` (hex, concatenated without the space). Keys of RSA-PSS, ML-DSA and SLH-DSA are produced by the library's secure random generator; their standard APIs cannot be seeded with a UID.

Warning: A key derived from a 10-digit UID has only about 33 bits of entropy (10^{10} possibilities) and can be brute-forced in seconds. UID-derived keys are used here **only for individualisation of the lab**; real private keys must always come from a cryptographically secure random generator (see Discussion Question 14).

6. Part A — Draw the Digital Signature Diagram (Pen and Paper)

This is the first task of the experiment and must be done before touching the computer. On a plain/ruled sheet, using pen/pencil and a ruler, draw the complete digital-signature process for a message sent from a sender to a receiver. **Append this hand-drawn sheet to your lab document at the time of submission.** Write your name, UID and date in the top-right corner.

Your diagram **must** contain (tick each item before you finish):

No.	Required element	Naming / convention to use	Done
1	Two parties and an insecure channel	Sender (Alice) on the left, Receiver (Bob) on the right, channel drawn in between	
2	Message and hash	Message M; hash function H( ); digest h = H(M)	
3	Sender's key pair	Private key SK_A (kept secret, drawn with a dashed arrow) and public key PK_A	
4	Signing step	S = Sign(SK_A, h); output pair (M ‖ S) where ‖ means concatenation	
5	Public-key distribution	Arrow showing PK_A reaching the receiver; mark that SK_A is never sent	
6	Receiver side	Received M′ and S; recompute h′ = H(M′); Verify(PK_A, h′, S)	
7	Decision	A decision box with two outputs: ACCEPT (valid) / REJECT (invalid)	
8	Attacker case	Mallory modifies M to M′ in the channel; show that the outcome is REJECT	

Tip: Use the same symbols (M, H, h, S, SK, PK, ‖) consistently throughout the experiment and in the report. Marks are given for correctness, naming and neatness.

7. Part B — Experiment Architecture

The experiment uses **one Docker image** and **two containers** (sender and receiver) connected by a **user-defined Docker network**. The sender creates keys, signs the document and serves the files over HTTP. The receiver downloads them and verifies the signature. Figure 1 shows the setup.

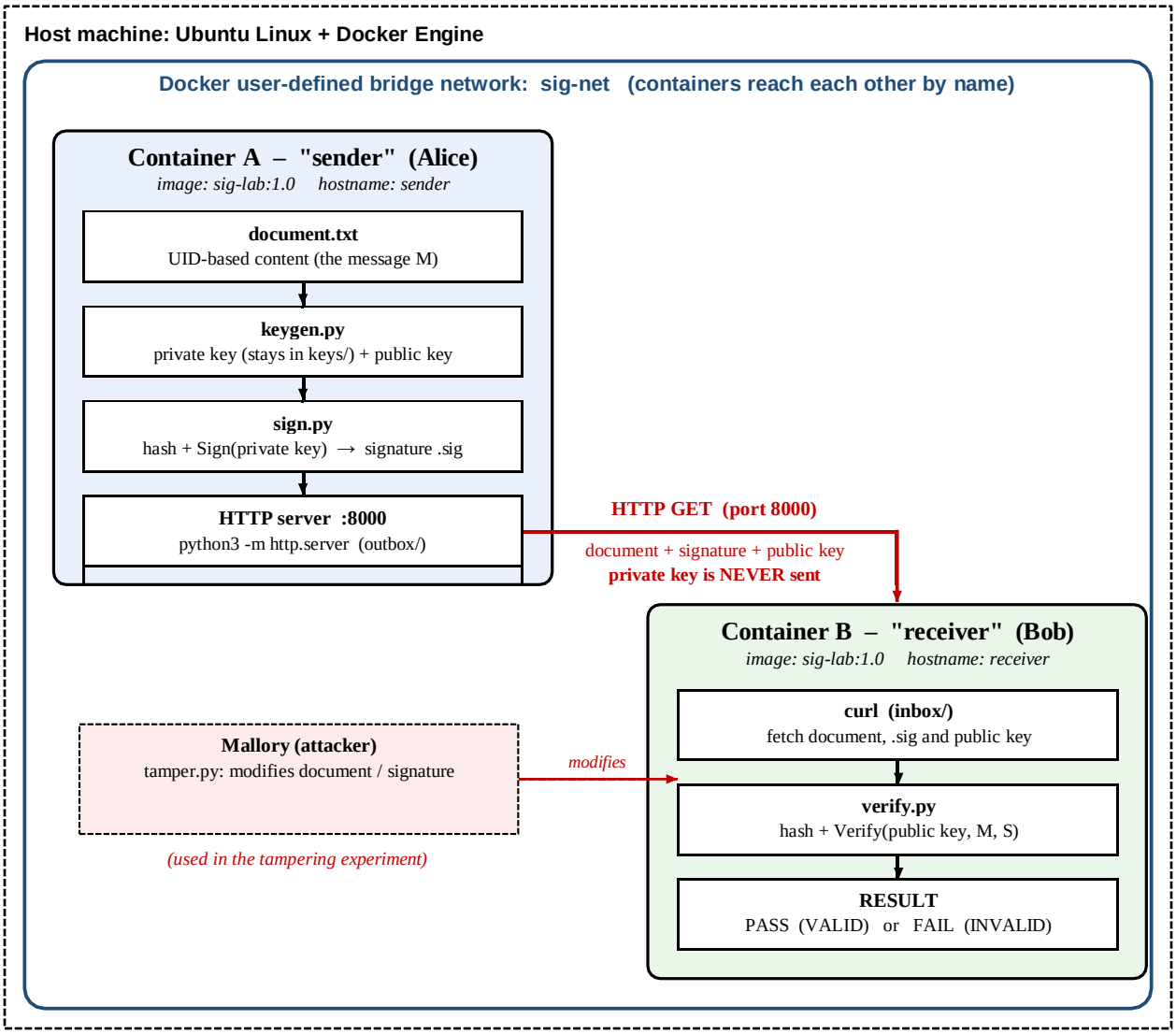


Figure 1: Docker-based sender–receiver setup. Only the document, the signature and the **public** key cross the network.

Programs to be written

File	Runs in	Purpose
common.py	both	UID handling, algorithm table, helper functions (file names, document creation, timing helper)
keygen.py <alg>	sender (and receiver for the wrong-key test)	Generate and save the key pair of the chosen algorithm
sign.py <alg>	sender	Create the document (once), sign it, write the signature to outbox/
verify.py <alg> [doc] [pubkey] [sig]	receiver	Verify a signature and print PASS / FAIL and the verification time
tamper.py <doc|sig> <file>	receiver	Modify one byte at the UID-derived position and save a modified copy
bench.py	sender (and receiver)	Benchmark key generation, signing and verification; write results.csv
plot.py	sender or host	Read the CSV files and draw the graphs

<alg> is one of: ecdsa, rsapss, mldsa, slhdsa.

8. Part C — Docker Environment Setup

All commands are typed in a terminal on an Ubuntu host (22.04 or 24.04). Lines starting with # are comments. Commands marked *host* run on your machine; commands marked *container* run inside a container.

Step C1: Install Docker (host)

```
sudo apt update
sudo apt install -y docker.io
sudo systemctl enable --now docker      # start Docker now and at every boot
sudo usermod -aG docker $USER           # allow docker without sudo
newgrp docker                           # apply the new group in this terminal
docker --version
docker run --rm hello-world             # test: prints a welcome message
```

Step C2: Create the working folders (host)

```
mkdir -p ~/sig-lab/sender ~/sig-lab/receiver
cd ~/sig-lab
```

These folders will be mounted inside the containers at /lab, so the programs you write and the results you produce stay on your computer even if a container is deleted.

Step C3: Write the Dockerfile (host)

Create ~/sig-lab/Dockerfile with exactly this content (it prepares Ubuntu 24.04 with Python, the *cryptography* library and the Open Quantum Safe library *liboqs-python*):

```
FROM ubuntu:24.04
ENV DEBIAN_FRONTEND=noninteractive
RUN apt-get update && apt-get install -y --no-install-recommends \
    python3 python3-pip python3-venv git cmake ninja-build build-essential \
    libssl-dev curl wget nano iproute2 iputils-ping tcpdump ca-certificates \
    && rm -rf /var/lib/apt/lists/*
RUN python3 -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
RUN pip install --no-cache-dir cryptography matplotlib pandas liboqs-python
# The first import makes liboqs-python download and build liboqs (a few minutes)
RUN python3 -c "import oqs; print(oqs.oqs_version())"
WORKDIR /lab
CMD ["sleep", "infinity"]
```

Step C4: Build the image (host)

```
cd ~/sig-lab
docker build -t sig-lab:1.0 .
docker images                           # sig-lab should be listed
```

> The build downloads packages and compiles liboqs; it can take 5–15 minutes. If it fails, check your internet connection, then retry with docker build --no-cache -t sig-lab:1.0 . and inform the instructor if the error continues.

Step C5: Create the Docker network (host)

```
docker network create sig-net
docker network ls                       # sig-net must appear (driver: bridge)
docker network inspect sig-net          # no containers attached yet
```

Step C6: Start the sender and receiver containers (host)

Replace `2024300020` by **your own UID**.

```
export STUDENT_UID=2024300020

docker run -dit --name sender   --hostname sender   --network sig-net \
   -e STUDENT_UID=$STUDENT_UID -v ~/sig-lab/sender:/lab   sig-lab:1.0

docker run -dit --name receiver --hostname receiver --network sig-net \
   -e STUDENT_UID=$STUDENT_UID -v ~/sig-lab/receiver:/lab sig-lab:1.0

docker ps                                # both containers must show "Up"
```

Option	Meaning
-d / -i / -t	run in background / keep input open / allocate a terminal
--name, --hostname	container name and the name by which other containers on sig-net can reach it
--network sig-net	attach the container to the user-defined network created in Step C5
-e STUDENT_UID=...	pass the UID into the container as an environment variable
-v host:container	mount a host folder inside the container (bind mount)

Step C7: Check connectivity between the containers

```
docker network inspect sig-net          # (host) both containers are now listed
docker exec -it sender bash             # (host) open a shell inside the sender
ping -c 3 receiver                      # (container) name resolution + reachability
echo $STUDENT_UID                       # (container) must print your UID
python3 -c "import oqs; print(oqs.get_enabled_sig_mechanisms())"
exit
```

Open **two terminals** for the rest of the lab: one with `docker exec -it sender bash` and one with `docker exec -it receiver bash`. Record the IP address of each container:

```
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' sender
docker inspect -f '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}' receiver
```

Step C8: Record the execution environment

Run-time measurements depend on the machine, so the environment must be recorded. Fill Table 1.

```
lscpu | grep -E "Model name|^CPU\(s\)"      # CPU model and number of threads
free -h | head -2                           # RAM
lsb_release -d ; uname -r                   # host OS and kernel
docker --version
docker exec sender pip list 2>/dev/null | grep -iE "cryptography|liboqs"
docker exec sender python3 -c "import oqs; print(oqs.oqs_version())"
```

Table 1: Execution environment

CPU model	Cores / threads	RAM	Host OS	Docker version	cryptography version	liboqs version

9. Part D — Common Module and Key Generation

Step D1: Find the exact algorithm names in liboqs (sender container)

```
python3 -c "import oqs; print([m for m in oqs.get_enabled_sig_mechanisms() if 'ML-DSA' in m or 'SLH' in m])"
```

Library versions name algorithms slightly differently (for example `ML-DSA-44` and `SLH_DSA_PURE_SHA2_128S` or `SLH-DSA-SHA2-128s`). **Use the exact string printed on your system.** If no SLH-DSA entry is printed, inform the instructor: liboqs must be rebuilt with SLH-DSA enabled.

Step D2: Write common.py (in /lab of both containers)

Edit with `nano common.py` (save: Ctrl+O, Enter; exit: Ctrl+X). It must contain:

- Read `STUDENT_UID` from `os.environ`; stop with an error message if it is missing or not 10 digits.
- A dictionary `ALGS` mapping `ecdsa, rsapss, mldsa, slhdsa` to their library names/parameters (use the strings found in Step D1).
- A function that builds the **document text** from the UID (format given in Section 5).
- A function that returns the **ECDSA private scalar** d derived from the UID (formula in Section 5; use `hashlib.sha256` and `int.from_bytes`).
- Helper functions returning the file paths, e.g. `keys/<alg>_private.key`, `outbox/<alg>_public.key`, `outbox/<alg>.sig`.
- A small timing helper that returns elapsed time in milliseconds using `time.perf_counter_ns()`.

Copy it to the receiver folder: `cp ~/sig-lab/sender/common.py ~/sig-lab/receiver/` (host).

Step D3: Write keygen.py (sender)

`python3 keygen.py <alg>` must generate a key pair for the chosen algorithm, write the private key to `keys/` and the public key to `outbox/`, and print the key sizes in bytes.

Algorithm	What to write in keygen.py	Hints (library calls)
ECDSA	Build the private key from the UID-derived scalar d on curve P-256. Save private key (PKCS#8 DER) and public key (SubjectPublicKeyInfo DER).	`ec.derive_private_key(d, ec.SECP256R1()),private_bytes(), public_key().public_bytes()`
RSA-PSS	Generate a 3072-bit RSA key with e = 65537. Save in the same DER formats.	`rsa.generate_private_key(public_exponent=65537, key_size=3072)`
ML-DSA	Create a signer object for the ML-DSA name, generate the key pair, save the raw public key and raw secret key.	`oqs.Signature(name), generate_keypair(), export_secret_key()`
SLH-DSA	Same procedure as ML-DSA with the SLH-DSA name.	as above

Use `help(oqs.Signature)` in the Python shell to see the exact methods of your version. Run:

```
cd /lab && mkdir -p keys outbox
python3 keygen.py ecdsa
python3 keygen.py rsapss
python3 keygen.py mldsa
python3 keygen.py slhdsa
ls -l keys outbox                       # note the size of every file
```

10. Part E — Signing the Document (sender)

`python3 sign.py <alg>` must: (1) create `outbox/document.txt` from the UID if it does not already exist, (2) read the document as bytes, (3) load the private key of the chosen algorithm, (4) sign, (5) write `outbox/<alg>.sig` and (6) print the signature size.

Algorithm	Signing parameters to use	Hints
ECDSA	SHA-256 with ECDSA; the signature is DER encoded	`load_der_private_key()`, `sign(data, ec.ECDSA(hashes.SHA256()))`
RSA-PSS	PSS padding, MGF1(SHA-256), salt length = SHA-256 digest length (32), hash SHA-256	`padding.PSS(mgf=padding.MGF1(hashes.SHA256()), salt_length=...)`
ML-DSA	Sign the whole document directly (no manual hashing)	`oqs.Signature(name, secret_key)` then `sign(data)`
SLH-DSA	Sign the whole document directly	as above

```
python3 sign.py ecdsa
python3 sign.py rsapss
python3 sign.py mldsa
python3 sign.py slhdsa                  # this one is noticeably slower
cat outbox/document.txt
ls -l outbox
sha256sum outbox/document.txt           # note this value; compare later on the receiver
```

11. Part F — Transfer over the Docker Network

The sender publishes the `outbox/` folder through a simple HTTP server and the receiver downloads the files by container name. **Only public information is placed in outbox/** (never the private key, which stays in `keys/`).

Sender terminal

```
cd /lab/outbox
python3 -m http.server 8000             # leave running; shows a log line per request
```

Receiver terminal

```
mkdir -p /lab/inbox && cd /lab/inbox
curl -O http://sender:8000/document.txt
for a in ecdsa rsapss mldsa slhdsa; do
    curl -sO http://sender:8000/${a}.sig
    curl -sO http://sender:8000/${a}_public.key
done
ls -l /lab/inbox
sha256sum document.txt                  # must equal the value noted on the sender
```

Observe the log lines printed by the sender's HTTP server: they show the **receiver's IP address** and the files requested. **Optional:** in a third terminal run `docker exec -it receiver tcpdump -nn -i eth0 port 8000 -c 40` before the downloads to see the packets that carry the signed data.

> **Think:** the public key was downloaded from the same untrusted channel as the document. Why is this not safe in a real system, and what is used in practice instead? (Discussion Question 10)

12. Part G — Signature Verification (receiver)

`python3 verify.py <alg> [document] [public_key] [signature]` must: read the document, the signature and the public key (optional arguments default to `inbox/document.txt`, `inbox/<alg>_public.key` and `inbox/<alg>.sig`), verify with the **same parameters used for signing**, measure the verification time, and print **PASS** or **FAIL**. Verification failure raises an exception in *cryptography* (`InvalidSignature`), while liboqs returns a boolean.

```
cd /lab
python3 verify.py ecdsa
python3 verify.py rsapss
python3 verify.py mldsa
python3 verify.py slhdsa
```

All four must print PASS. Record the sizes (use `stat -c %s <file>`) in Table 2.

Table 2: Key, signature and transmission sizes (bytes)

Algorithm	Public key	Private key	Signature	Document	Transmitted (doc + sig + public key)
ECDSA					
RSA-PSS					
ML-DSA					
SLH-DSA					

Sanity check (from the standards, for the raw/unencoded keys): ML-DSA-44: public key 1312, secret key 2560, signature 2420 bytes. SLH-DSA-SHA2-128s: public key 32, secret key 64, signature 7856 bytes. Classical keys saved as DER will be slightly larger than the raw values.

13. Part H — Tampering Detection

Write `tamper.py <doc|sig> <file>`. It must read the file, change **exactly one character/bit** at index = UID mod (file length), and save the result as `<file>.tampered` (the original file must not be overwritten). Then run all four algorithms for each test case:

```
cd /lab
python3 tamper.py doc inbox/document.txt         # -> inbox/document.txt.tampered
python3 verify.py ecdsa inbox/document.txt.tampered      # Test 2: modified document

python3 tamper.py sig inbox/ecdsa.sig            # -> inbox/ecdsa.sig.tampered
python3 verify.py ecdsa inbox/document.txt inbox/ecdsa_public.key inbox/ecdsa.sig.tampered
                                                 # Test 3: modified signature

mkdir -p keys outbox && python3 keygen.py ecdsa  # Test 4: receiver makes ITS OWN key pair
python3 verify.py ecdsa inbox/document.txt outbox/ecdsa_public.key
                                                 # receiver's public key, sender's signature
```

> In Test 4 the receiver's key generation writes into the receiver's own `/lab` folder, so it does not disturb the files downloaded from the sender. Take care not to confuse the two public keys.

Table 3: Verification results

Test case	ECDSA	RSA-PSS	ML-DSA	SLH-DSA	Expected
1. Original document + original signature					PASS
2. One character of document changed					FAIL
3. One bit of signature changed					FAIL
4. Correct document + signature, wrong public key					FAIL

Also write down the original and the modified document text (the changed character) in your report.

14. Part I — Performance Benchmarking

Write `bench.py` to measure, for each of the four algorithms, the time of **key generation**, **signing** and **verification** of the 1024-byte UID-seeded message (Section 5). The benchmark measures **only the cryptographic operation**; Docker start-up, file I/O and network transfer must **not** be inside the timed region.

- Perform **5 warm-up** runs (not recorded) for every operation.
- Repeat: key generation **20** times, signing **100** times, verification **100** times (key generation is slow for RSA; signing is slow for SLH-DSA).
- Time each call individually with `time.perf_counter_ns()` and convert to milliseconds.
- Compute **mean, standard deviation, minimum and maximum** for every algorithm and operation.
- Write `results.csv` with columns: `uid, algorithm, operation, runs, mean_ms, std_ms, min_ms, max_ms`; also store public key, private key and signature sizes.
- Verify inside the benchmark that signatures from the last signing loop pass, so that you do not time failed operations.

```
# on the sender
cd /lab && python3 bench.py
cat results.csv

# repeat on the receiver to compare two containers on the same machine
docker exec -it receiver bash -c "cd /lab && python3 bench.py"

# run it 3 times on the sender and keep the median run (rename files results_run1.csv ...)
# optional: restrict the container to one CPU core and run again
docker update --cpus 1 sender
```

Table 4: Timing results (milliseconds, mean ± standard deviation over the runs)

Algorithm	Key generation	Signing	Verification	Sign / Verify ratio
ECDSA				
RSA-PSS				
ML-DSA				
SLH-DSA				

15. Graphical Analysis

Write `plot.py` (matplotlib; use pandas or `csv` to read results.csv) and generate the graphs below. Save them as PNG files in `/lab/graphs/`, copy them to the host with `docker cp sender:/lab/graphs ./graphs` and paste them in your report. Every graph needs a title, axis labels with units, a legend where required, and the execution environment in the caption. Use a **logarithmic y-axis** where values differ by orders of magnitude.

Graph	Type	Content
Graph 1	Grouped bar	Public-key size and private-key size (bytes) for the four algorithms
Graph 2	Bar	Signature size (bytes) for the four algorithms

Graph	Type	Content
Graph 3	Bar with error bars	Average signing time (ms), error bar = standard deviation
Graph 4	Bar with error bars	Average verification time (ms)
Graph 5	Bar	Average key-generation time (ms)
Graph 6	Stacked bar	Total transmitted bytes = document + signature + public key
Graph 7 (extra)	Grouped bar	Sign, verify and key-generation time of every algorithm relative to ECDSA (ECDSA = 1)

16. Part J — Comparative Analysis

Table 5: Relative comparison with ECDSA as the baseline (divide your measured value by the ECDSA value)

Parameter	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Public key size	1.0			
Signature size	1.0			
Signing time	1.0			
Verification time	1.0			
Key generation time	1.0			
Total transmitted bytes	1.0			

Table 6: Property comparison (fill from your observations and from the standards)

Property	ECDSA	RSA-PSS	ML-DSA	SLH-DSA
Standard				
Security parameter / category				
Mathematical basis				
Resistant to Shor’s algorithm?				
Public key size (small / medium / large)				
Signature size				
Fastest operation				
Slowest operation				
Maturity / deployment				
Main advantage				
Main disadvantage				

Table 7: Which algorithm would you choose? Choose one algorithm for each scenario and justify in one line.

Scenario	Chosen algorithm	Justification
Signing a firmware image for a device that stays in the field for 15 years		
Authenticating a short sensor message on a very low-bandwidth link		
Signing a TLS server certificate in 2026 on a general web server		
Root key of a certificate authority that must remain trusted for 30 years		
High-volume server that verifies millions of signatures per second		

17. Discussion Questions

Conceptual

1. What are the three algorithms of a digital-signature scheme? Which key is used in which algorithm?
2. What is the difference between a digital signature and a MAC (for example HMAC)? Why does the signature allow non-repudiation?
3. Why is the private key never transferred in this experiment? What would happen if it were?
4. **Why did post-quantum cryptography come into the picture?** Explain Shor's algorithm's effect on RSA and ECDSA, and why a signature that is valid today may be dangerous to rely on in 20 years.

Analytical (use your measured data)

5. Which algorithm had the smallest and which the largest signature? Express the difference as a ratio and explain the cause.
6. Which algorithm was fastest to sign and which to verify? Why is SLH-DSA signing much slower than ML-DSA signing?
7. SLH-DSA has a tiny public key (32 bytes) but a large signature. ML-DSA has a larger public key but a smaller signature. Discuss the trade-off for a system that sends one public key but millions of signatures.
8. The public key was fetched over the same untrusted channel as the document. Explain how a man-in-the-middle could defeat the scheme and how certificates and a PKI solve this.

Application and evaluation

9. Is ML-DSA or SLH-DSA "better" than ECDSA? Answer using your data, and consider size, speed, maturity and quantum security. Is there a single winner?

10. Why do many experts recommend **hybrid** signatures (a classical plus a post-quantum signature) during the transition period?
11. Does the Docker network affect the cryptographic measurements? How did you keep Docker, the network and file I/O out of your timings, and what other factors (CPU frequency, background load, container CPU limit) can still influence them?

18. Final Conclusion

Students shall write a **200–250 word** conclusion addressing:

- How a digital signature gives integrity and authentication, and the role of the private and public keys.
- What happened to the signature when the document, the signature or the key was changed.
- Size and speed differences you observed among RSA-PSS, ECDSA, ML-DSA and SLH-DSA.
- Why post-quantum signatures are needed and what price (size, speed) is paid for them.
- Which algorithm you would choose for today's applications and which for long-term security, with reasons.
- What you learned about Docker containers and networks.

19. Submission

Students have to submit (in this order):

- **Hand-drawn digital-signature diagram** (Part A) with name, UID and date
- Table 1: execution environment, plus screenshots of `docker ps`, `docker network inspect sig-net` and the ping test
- Screenshots of key generation, signing, transfer (including the sender's HTTP log) and verification for all four algorithms
- Table 2 (sizes), Table 3 (tampering results with original and modified text), Table 4 (timings)
- Seven graphs (Section 15), Tables 5, 6 and 7
- Answers to all discussion questions
- 200–250 word conclusion
- Source code of common.py, keygen.py, sign.py, verify.py, tamper.py, bench.py, plot.py and the Dockerfile
- results.csv files (sender, receiver) and your UID

Appendix A — Useful Docker and Linux Commands

Task	Command
List running / all containers	`docker ps` / `docker ps -a`
Open a shell in a running container	`docker exec -it sender bash`
Run one command in a container	`docker exec sender ls -l /lab/outbox`
Copy a file container → host	`docker cp sender:/lab/results.csv ./results_sender.csv`
Show logs of a container	`docker logs receiver`
Stop / start / restart	`docker stop sender` / `docker start sender` / `docker restart sender`
Fix root-owned files created by a container	`sudo chown -R $USER:$USER ~/sig-lab`
File size in bytes	`stat -c %s file` or `wc -c < file`
View a binary file	`xxd file | head`
Clean up after the experiment	`docker rm -f sender receiver; docker network rm sig-net`
Remove the image (frees disk space)	`docker rmi sig-lab:1.0`

Troubleshooting

- `permission denied` on the Docker socket: run `newgrp docker` or log out and in after `usermod`.
- `curl: Could not resolve host: sender`: the containers are not on the same user-defined network. Check with `docker network inspect sig-net`.
- `Connection refused` on port 8000: the HTTP server in the sender is not running, or you started it in a different folder.
- `ModuleNotFoundError: oqs` or an error loading liboqs: you are not inside the `sig-lab` container, or the image build did not finish.
- Verification fails for a valid signature: check that signing and verification use identical hash, padding and parameter set, and that the document file was not changed or re-encoded (compare `sha256sum` on both sides).
- SLH-DSA is slow: this is expected. Reduce nothing; just wait. Do not run other heavy programs during benchmarking.

Optional extensions: repeat the experiment with ML-DSA-65/87, SLH-DSA-SHA2-128f (compare with the 's' variant), RSA-2048 vs RSA-3072, or P-384, and discuss the size–speed trade-off.